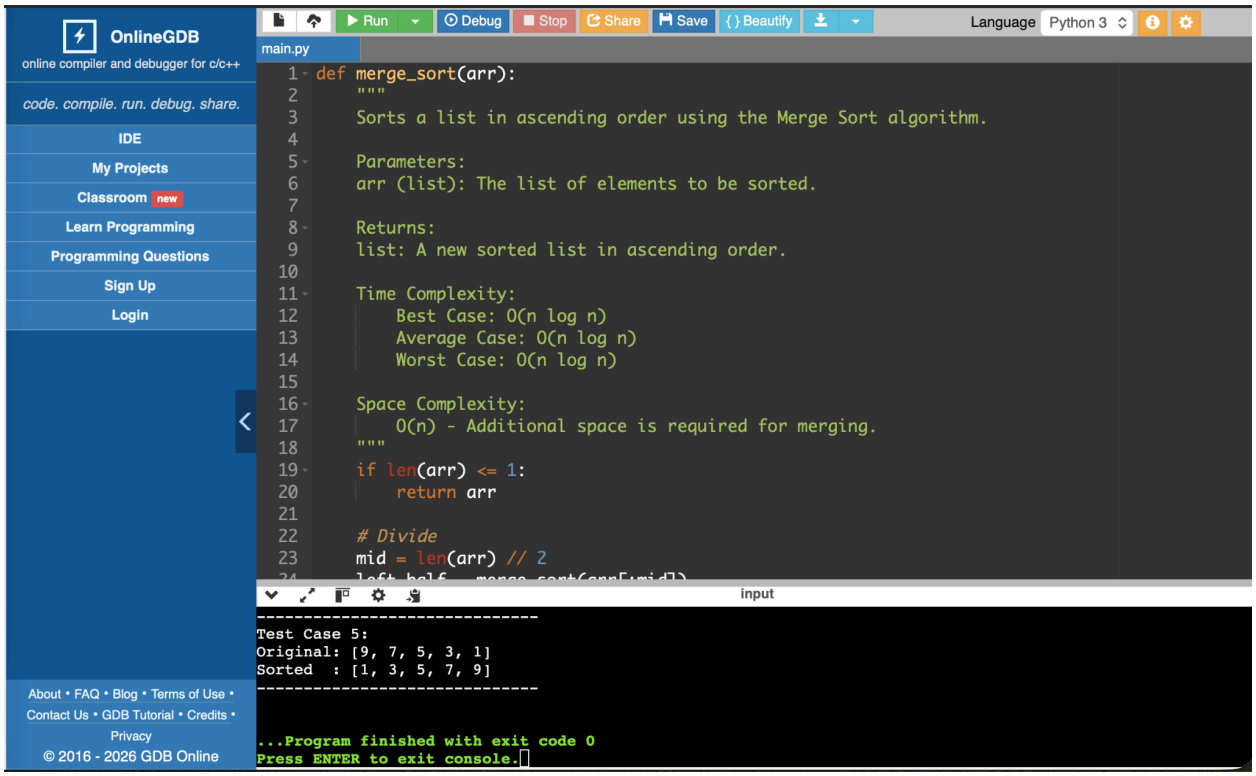
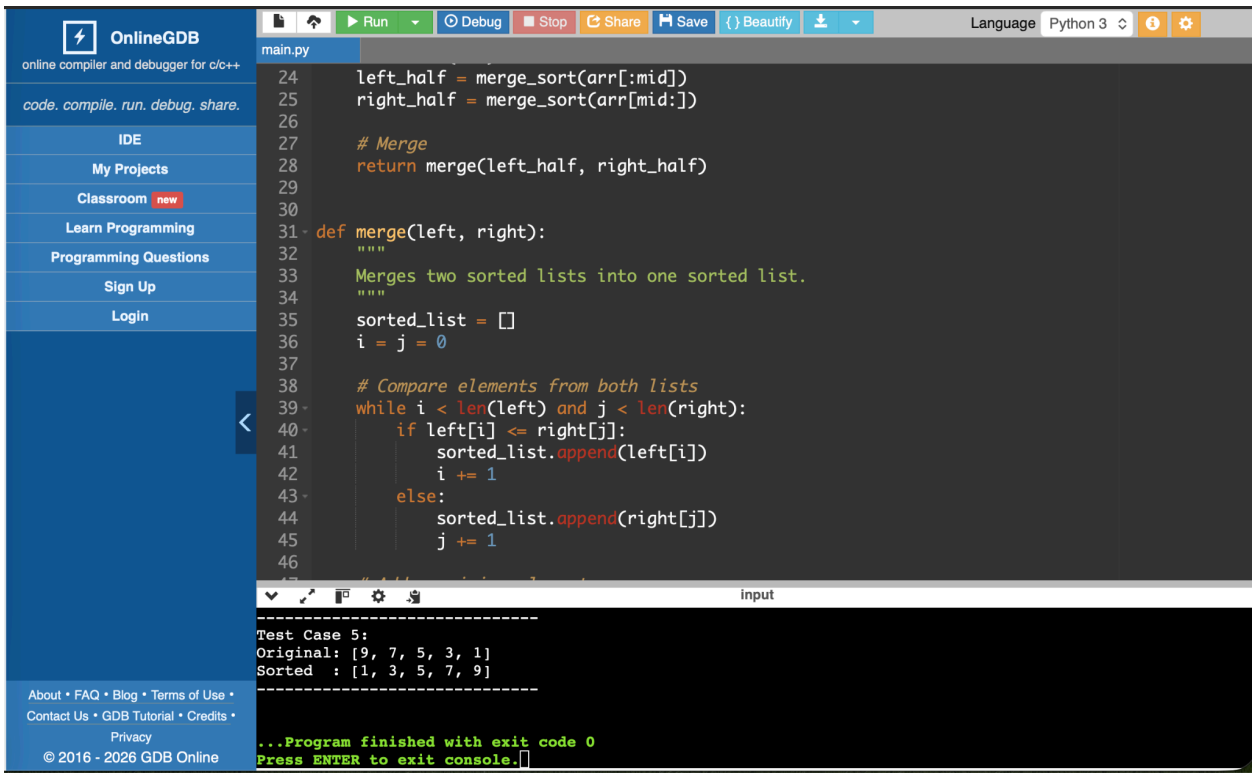
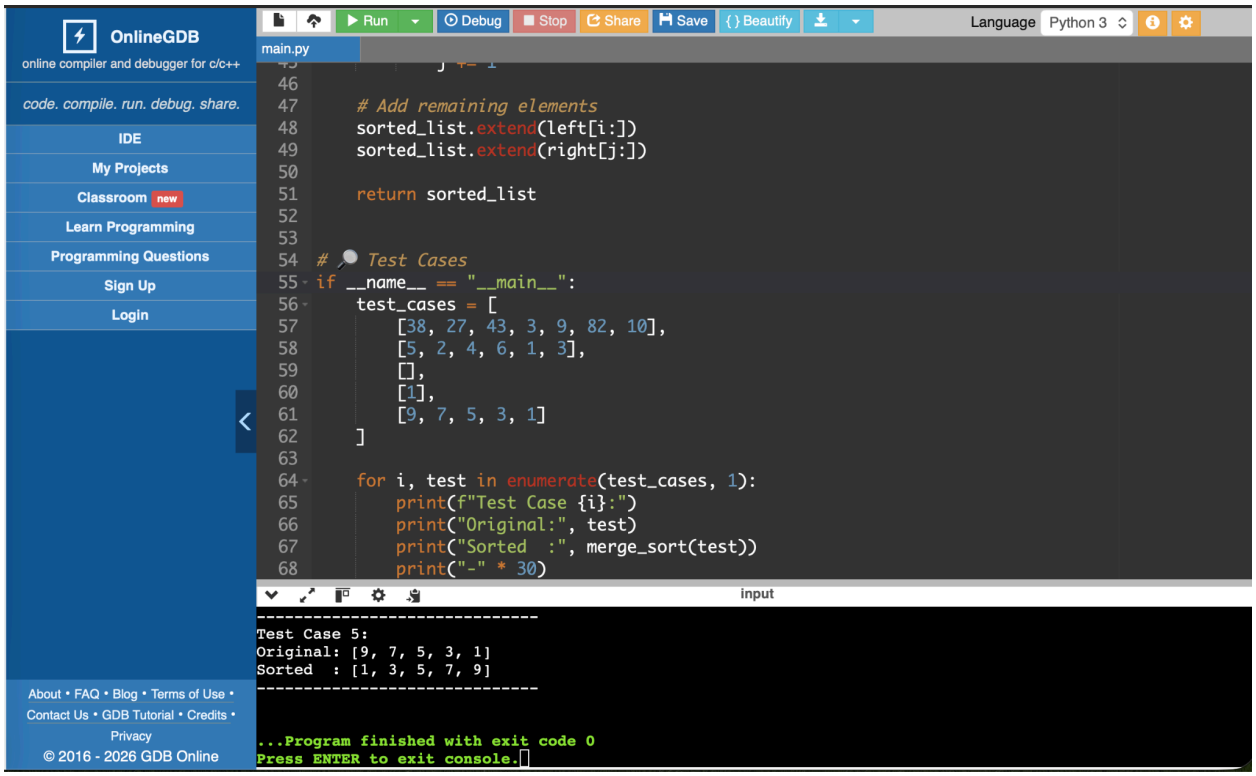
2303A52416
batch=35
task Description #1 (Sorting – Merge Sort Implementation)
• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
• Instructions:
o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
o Ask AI to include time complexity and space complexity in the function docstring.
o Verify the generated code with test cases.
• Expected Output:
o A functional Python script implementing Merge Sort with proper documentation.
Code

```
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list in ascending order.

    Time Complexity:
        Best Case: O(n log n)
        Average Case: O(n log n)
        Worst Case: O(n log n)

    Space Complexity:
        O(n) - Additional space is required for merging.
    """
    if len(arr) <= 1:
        return arr

    # Divide
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    # Merge
    return merge(left_half, right_half)
```

```
def merge(left, right):
    """
    Merges two sorted lists into one sorted list.
    """
    sorted_list = []
    i = j = 0

    # Compare elements from both lists
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1

    # Add remaining elements
    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])

    return sorted_list


# 🔎 Test Cases
if __name__ == "__main__":
    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 4, 6, 1, 3],
        [],
        [1],
        [9, 7, 5, 3, 1]
    ]

    for i, test in enumerate(test_cases, 1):
        print(f"Test Case {i}:")
        print("Original:", test)
        print("Sorted  :", merge_sort(test))
        print("-" * 30)
```


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
def merge_sort(arr):
"""
Sorts a list in ascending order using the Merge Sort algorithm.
Parameters:
arr (list): The list of elements to be sorted.
Returns:
list: A new sorted list in ascending order.
Time Complexity:
Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n log n)
Space Complexity:
O(n) - Additional space is required for merging.
"""
if len(arr) <= 1:
return arr
# Divide
mid = len(arr) // 2
input
Test Case 5:
Original: [9, 7, 5, 3, 1]
Sorted : [1, 3, 5, 7, 9]
...Program finished with exit code 0
Press ENTER to exit console.


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
left_half = merge_sort(arr[:mid])
right_half = merge_sort(arr[mid:])
# Merge
return merge(left_half, right_half)
def merge(left, right):
"""
Merges two sorted lists into one sorted list.
"""
sorted_list = []
i = j = 0
# Compare elements from both lists
while i < len(left) and j < len(right):
if left[i] <= right[j]:
sorted_list.append(left[i])
i += 1
else:
sorted_list.append(right[j])
j += 1
input
Test Case 5:
Original: [9, 7, 5, 3, 1]
Sorted : [1, 3, 5, 7, 9]
...Program finished with exit code 0
Press ENTER to exit console.



Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

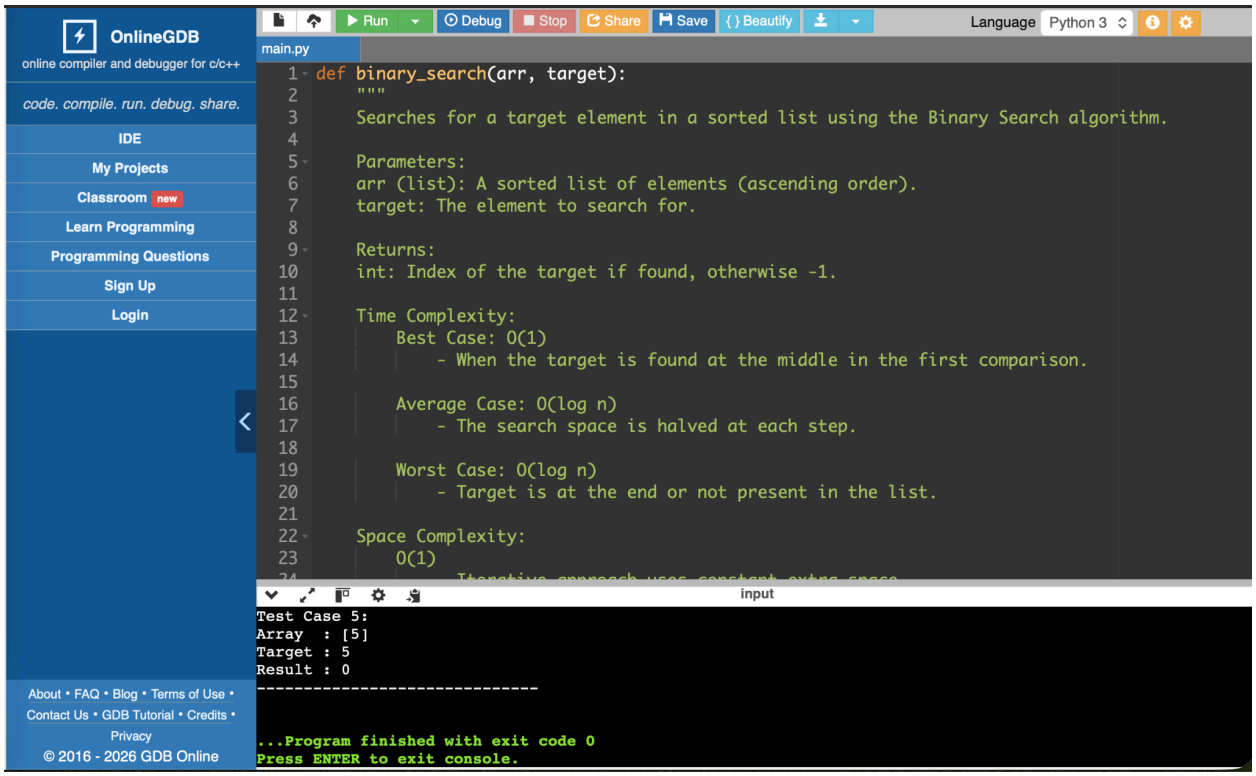
o Python code implementing binary search with AI-generated comments and docstrings.

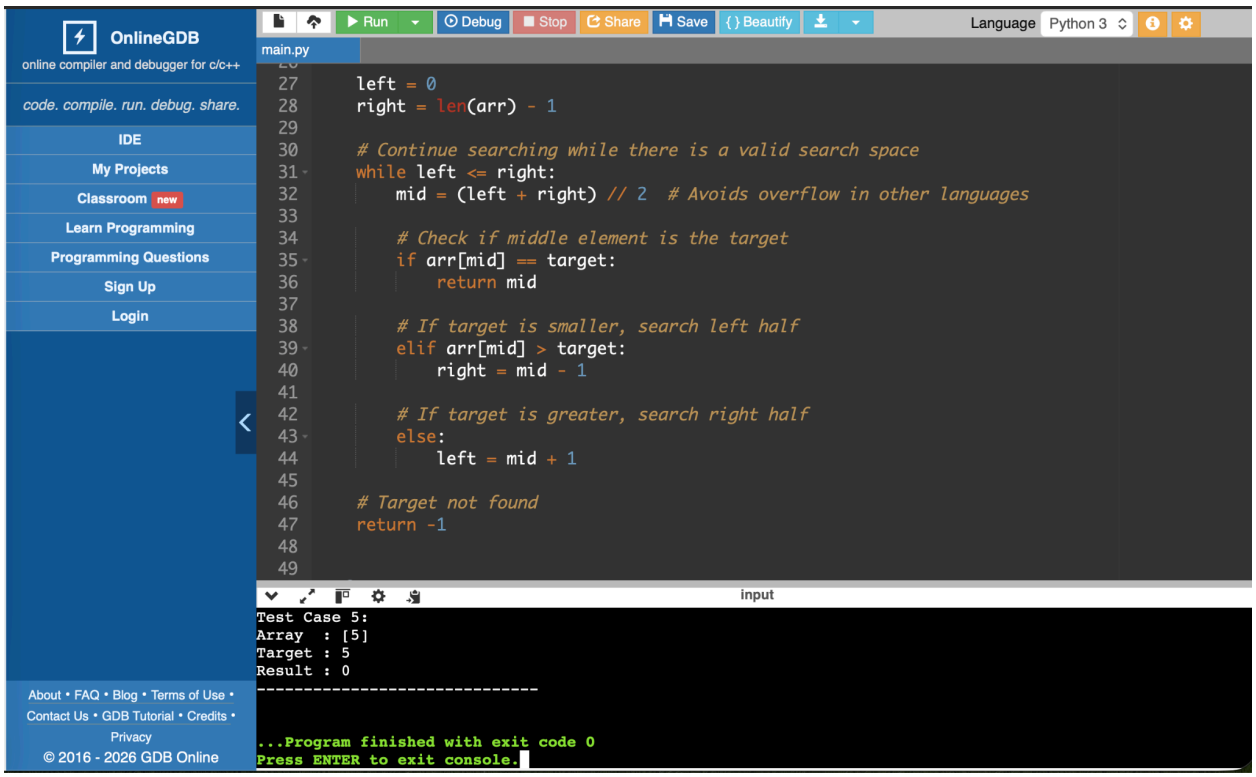
Code

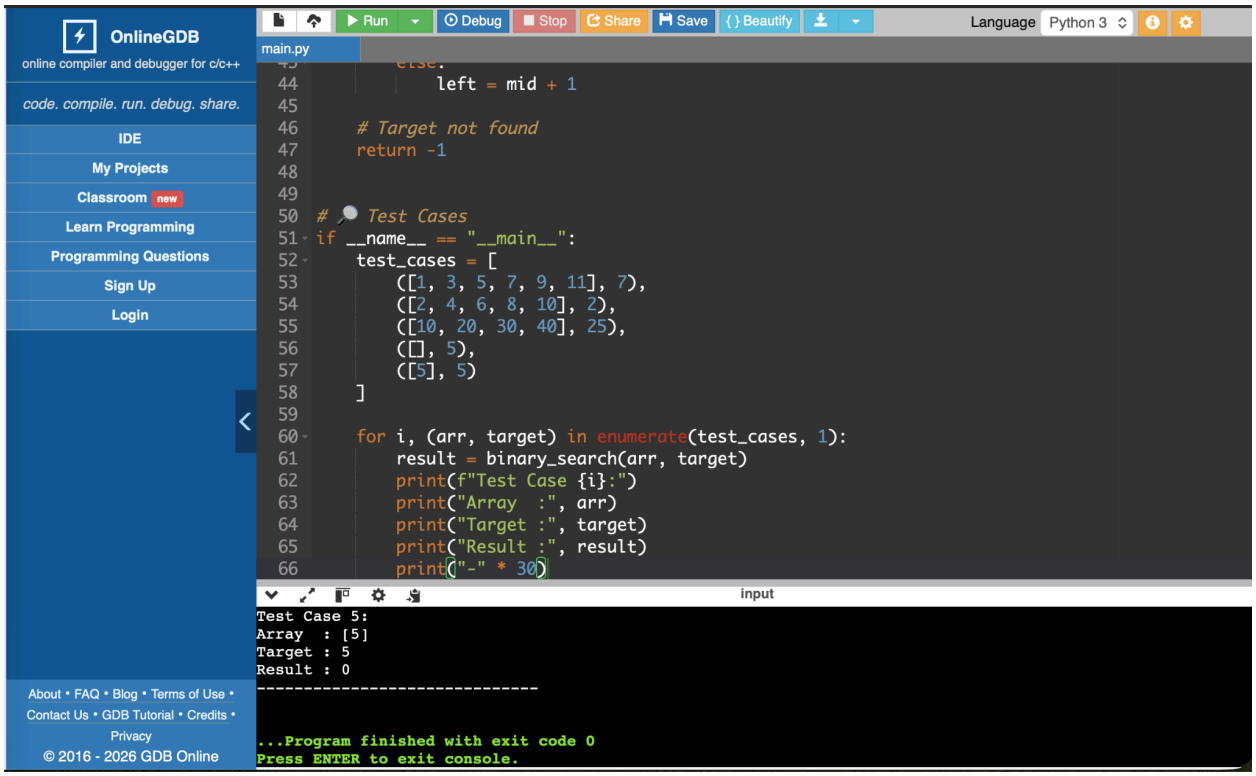
```
def binary_search(arr, target):
    """
    Searches for a target element in a sorted list using the Binary Search algorithm.

    Parameters:
    arr (list): A sorted list of elements (ascending order).
    target: The element to search for.
```

```
    Returns:
    int: Index of the target if found, otherwise -1.

    Time Complexity:
        Best Case: O(1)
            - When the target is found at the middle in the first comparison.

        Average Case: O(log n)
            - The search space is halved at each step.

        Worst Case: O(log n)
            - Target is at the end or not present in the list.

    Space Complexity:
        O(1)
            - Iterative approach uses constant extra space.
    """

    left = 0
    right = len(arr) - 1

    # Continue searching while there is a valid search space
    while left <= right:
        mid = (left + right) // 2  # Avoids overflow in other languages

        # Check if middle element is the target
        if arr[mid] == target:
            return mid

        # If target is smaller, search left half
        elif arr[mid] > target:
            right = mid - 1

        # If target is greater, search right half
        else:
            left = mid + 1

    # Target not found
    return -1


# 🔎 Test Cases
if __name__ == "__main__":
    test_cases = [
```

```
    ([1, 3, 5, 7, 9, 11], 7),
    ([2, 4, 6, 8, 10], 2),
    ([10, 20, 30, 40], 25),
    ([], 5),
    ([5], 5)
]

for i, (arr, target) in enumerate(test_cases, 1):
    result = binary_search(arr, target)
    print(f"Test Case {i}:")
    print("Array  :", arr)
    print("Target :", target)
    print("Result :", result)
    print("-" * 30)
```


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
def binary_search(arr, target):
"""
Searches for a target element in a sorted list using the Binary Search algorithm.
Parameters:
arr (list): A sorted list of elements (ascending order).
target: The element to search for.
Returns:
int: Index of the target if found, otherwise -1.
Time Complexity:
Best Case: O(1)
- When the target is found at the middle in the first comparison.
Average Case: O(log n)
- The search space is halved at each step.
Worst Case: O(log n)
- Target is at the end or not present in the list.
Space Complexity:
O(1)
input
Test Case 5:
Array : [5]
Target : 5
Result : 0
...Program finished with exit code 0
Press ENTER to exit console.


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
left = 0
right = len(arr) - 1
# Continue searching while there is a valid search space
while left <= right:
mid = (left + right) // 2 # Avoids overflow in other languages
# Check if middle element is the target
if arr[mid] == target:
return mid
# If target is smaller, search left half
elif arr[mid] > target:
right = mid - 1
# If target is greater, search right half
else:
left = mid + 1
# Target not found
return -1
input
Test Case 5:
Array : [5]
Target : 5
Result : 0
...Program finished with exit code 0
Press ENTER to exit console.



Task Description #3 (Real-Time Application – Inventory Management System)

• Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

• Task:

o Use AI to suggest the most efficient search and sort algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update frequency, and performance requirements.

• Expected Output:

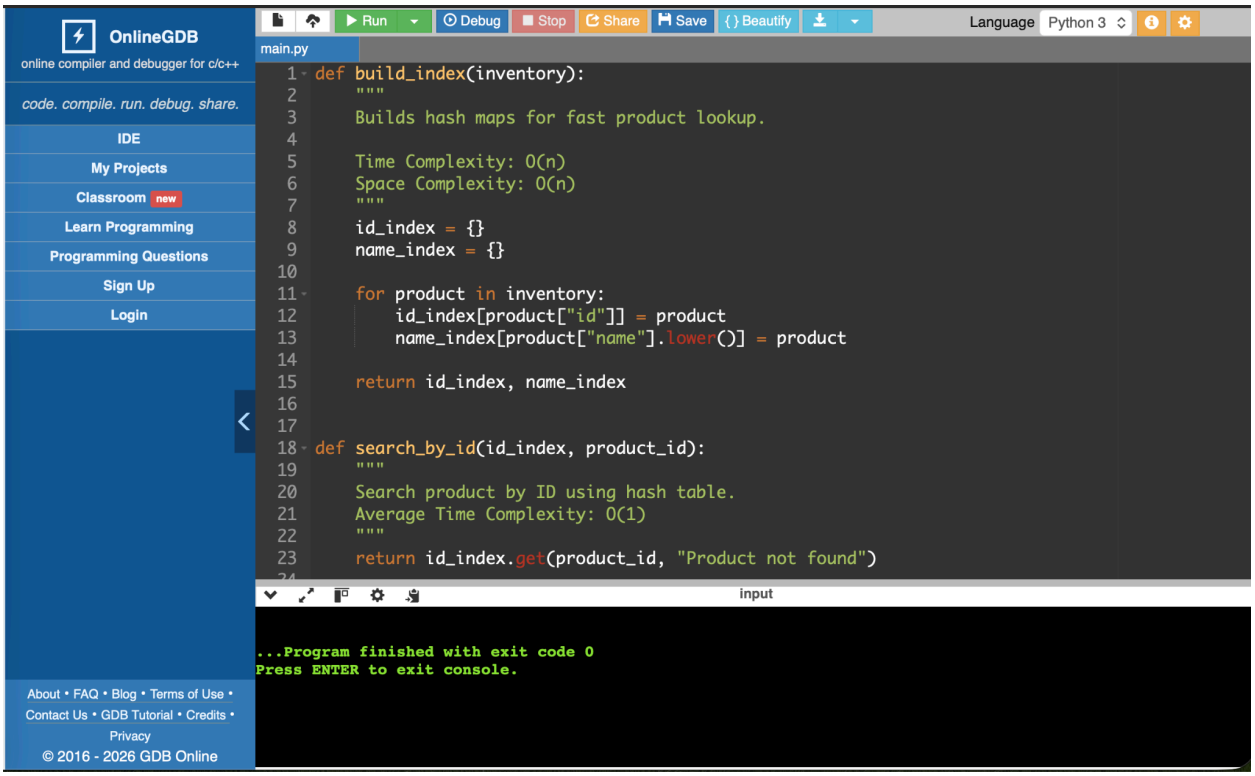
o A table mapping operation → recommended algorithm → justification.

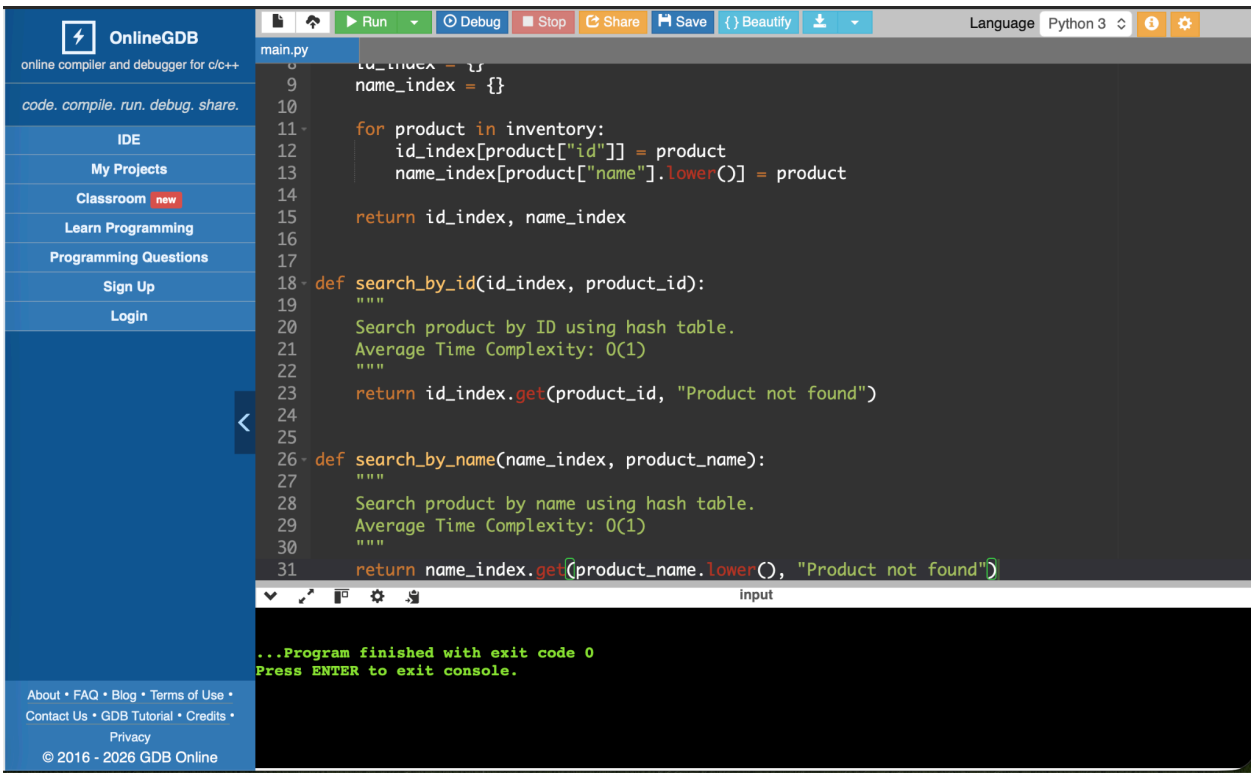
o Working Python functions for searching and sorting the inventory.

Code

```
def build_index(inventory):
    """
    Builds hash maps for fast product lookup.

    Time Complexity: O(n)
```

```
    Space Complexity: O(n)
    """
    id_index = {}
    name_index = {}

    for product in inventory:
        id_index[product["id"]] = product
        name_index[product["name"].lower()] = product

    return id_index, name_index


def search_by_id(id_index, product_id):
    """
    Search product by ID using hash table.
    Average Time Complexity: O(1)
    """
    return id_index.get(product_id, "Product not found")


def search_by_name(name_index, product_name):
    """
    Search product by name using hash table.
    Average Time Complexity: O(1)
    """
    return name_index.get(product_name.lower(), "Product not found")
```





Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill

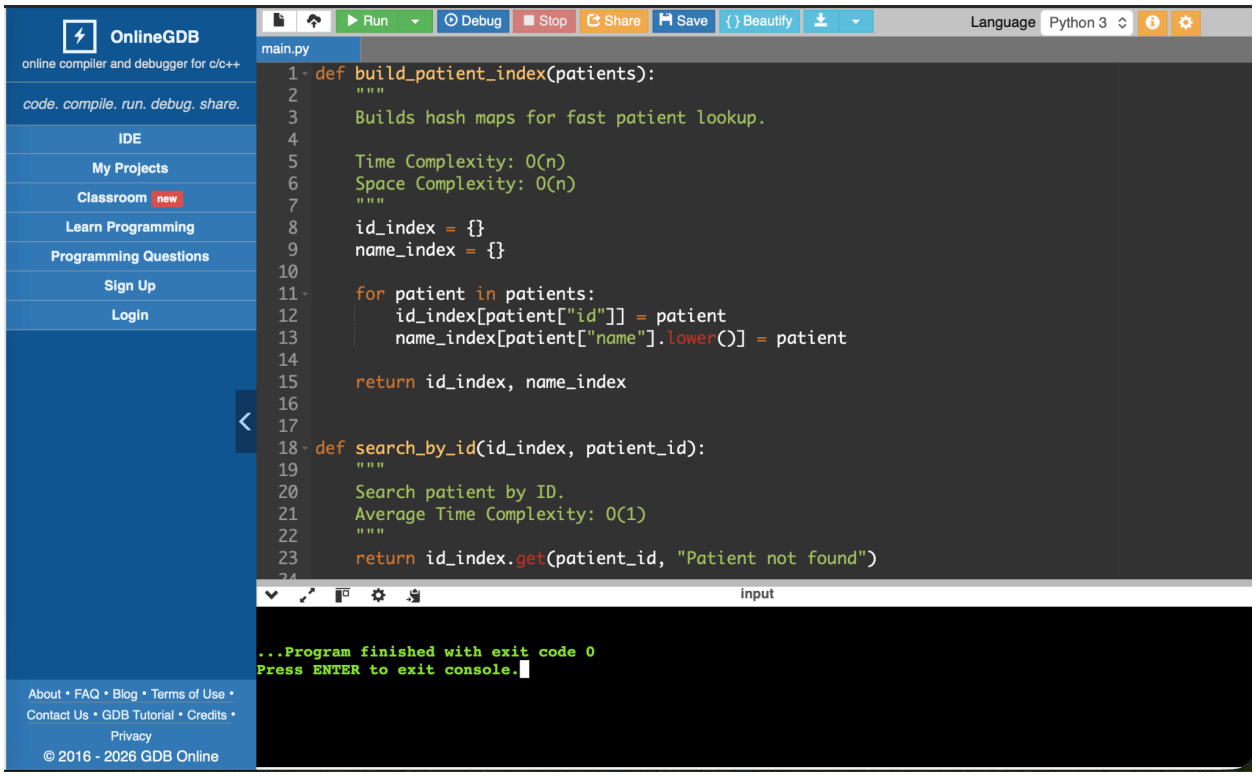
amount. Doctors and staff need to:
1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.

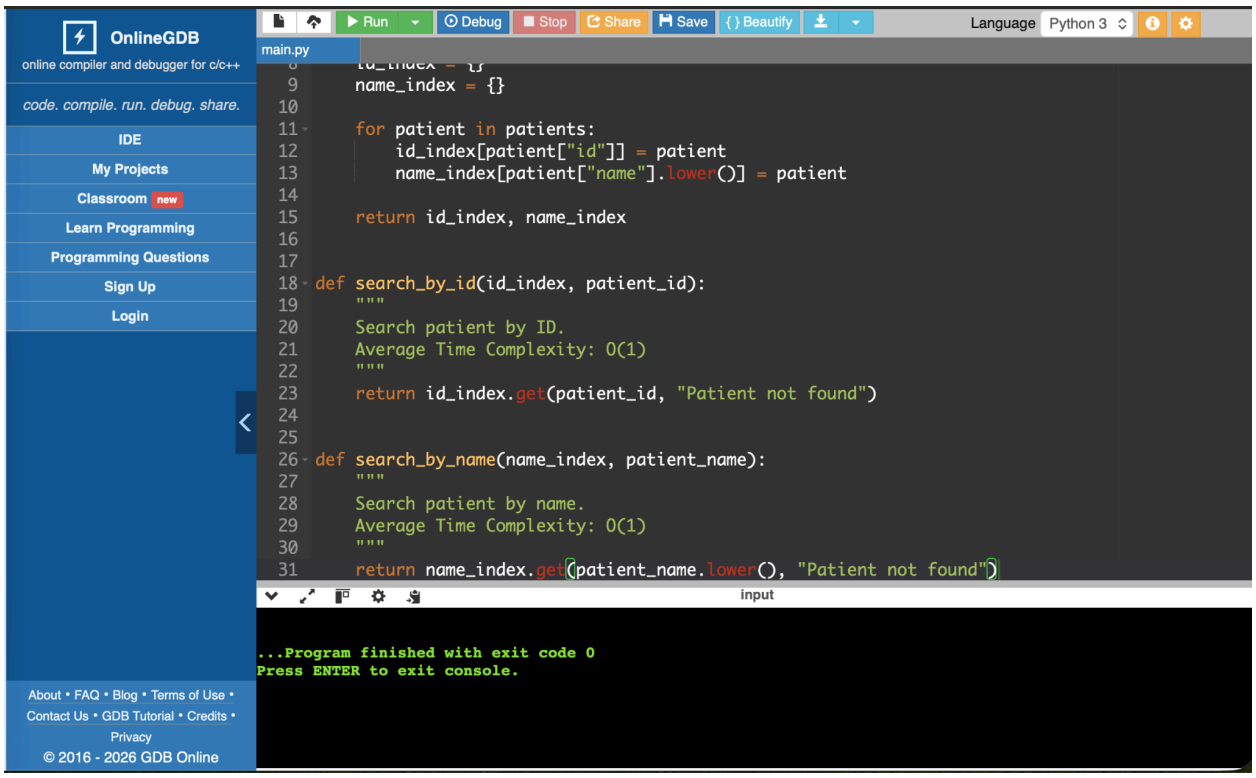
Student Task

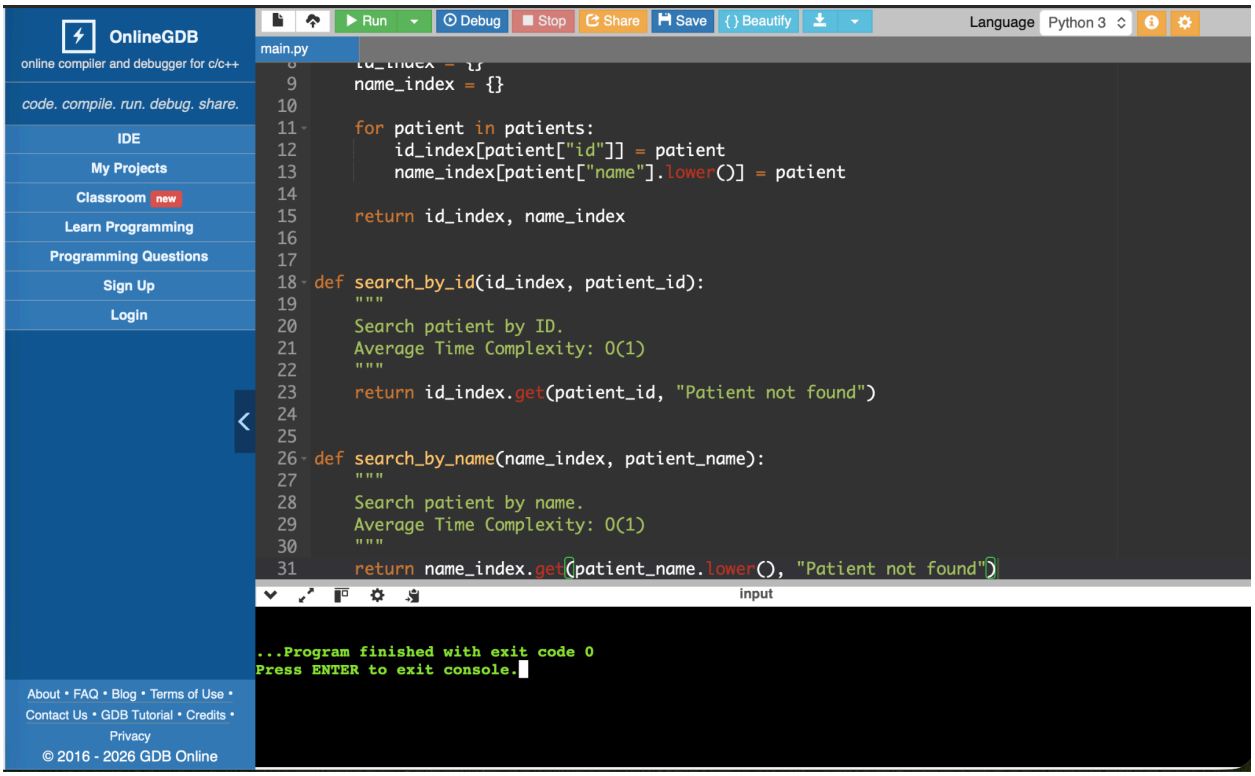
• Use AI to recommend suitable searching and sorting algorithms.
• Justify the selected algorithms in terms of efficiency and suitability.
• Implement the recommended algorithms in Python.

Code

```
def build_patient_index(patients):
    """
    Builds hash maps for fast patient lookup.

    Time Complexity: O(n)
    Space Complexity: O(n)
    """
    id_index = {}
    name_index = {}

    for patient in patients:
        id_index[patient["id"]] = patient
        name_index[patient["name"].lower()] = patient

    return id_index, name_index


def search_by_id(id_index, patient_id):
    """
    Search patient by ID.
    Average Time Complexity: O(1)
    """
    return id_index.get(patient_id, "Patient not found")


def search_by_name(name_index, patient_name):
    """
    Search patient by name.
    Average Time Complexity: O(1)
    """
    return name_index.get(patient_name.lower(), "Patient not found")
```


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
1 def build_patient_index(patients):
2 """
3 Builds hash maps for fast patient lookup.
4
5 Time Complexity: O(n)
6 Space Complexity: O(n)
7 """
8 id_index = {}
9 name_index = {}
10
11 for patient in patients:
12 id_index[patient["id"]] = patient
13 name_index[patient["name"].lower()] = patient
14
15 return id_index, name_index
16
17
18 def search_by_id(id_index, patient_id):
19 """
20 Search patient by ID.
21 Average Time Complexity: O(1)
22 """
23 return id_index.get(patient_id, "Patient not found")
input
...Program finished with exit code 0
Press ENTER to exit console.


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
9 name_index = {}
10
11 for patient in patients:
12 id_index[patient["id"]] = patient
13 name_index[patient["name"].lower()] = patient
14
15 return id_index, name_index
16
17
18 def search_by_id(id_index, patient_id):
19 """
20 Search patient by ID.
21 Average Time Complexity: O(1)
22 """
23 return id_index.get(patient_id, "Patient not found")
24
25
26 def search_by_name(name_index, patient_name):
27 """
28 Search patient by name.
29 Average Time Complexity: O(1)
30 """
31 return name_index.get(patient_name.lower(), "Patient not found")
input
...Program finished with exit code 0
Press ENTER to exit console.



Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.
2. Sort students based on marks to generate rank lists.

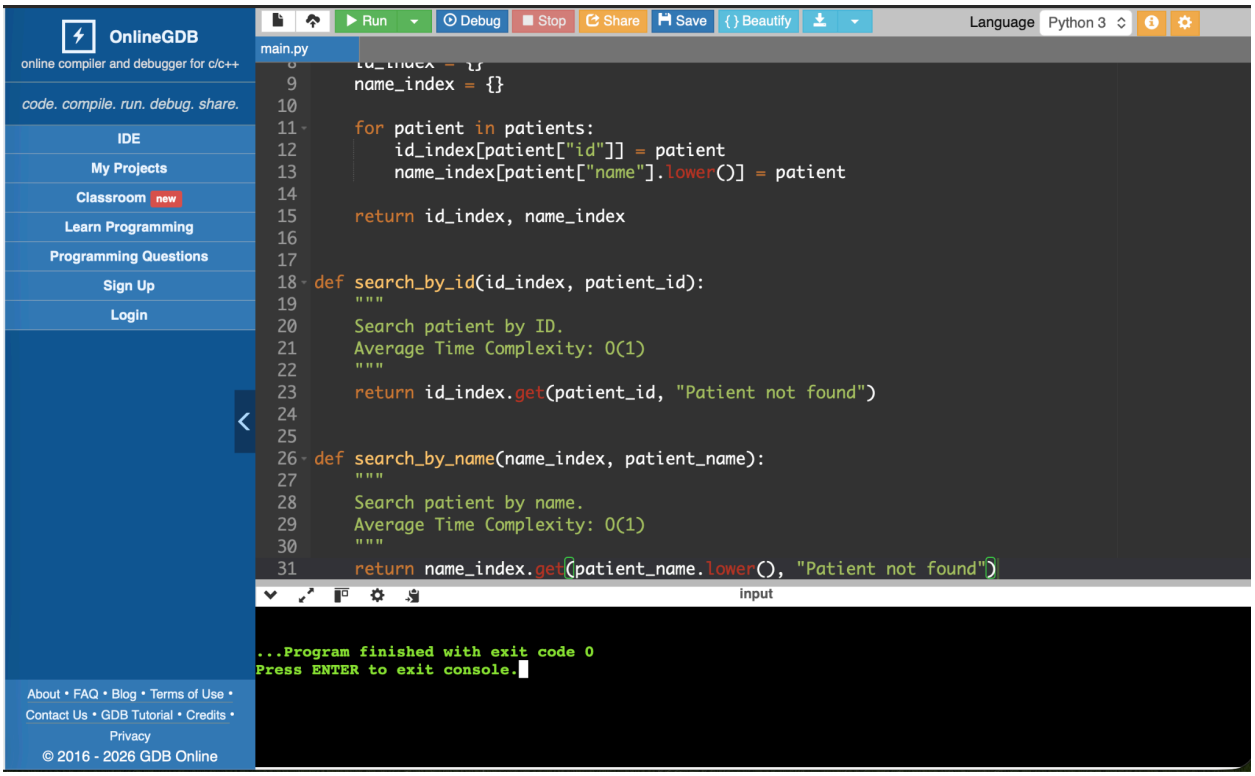
Student Task

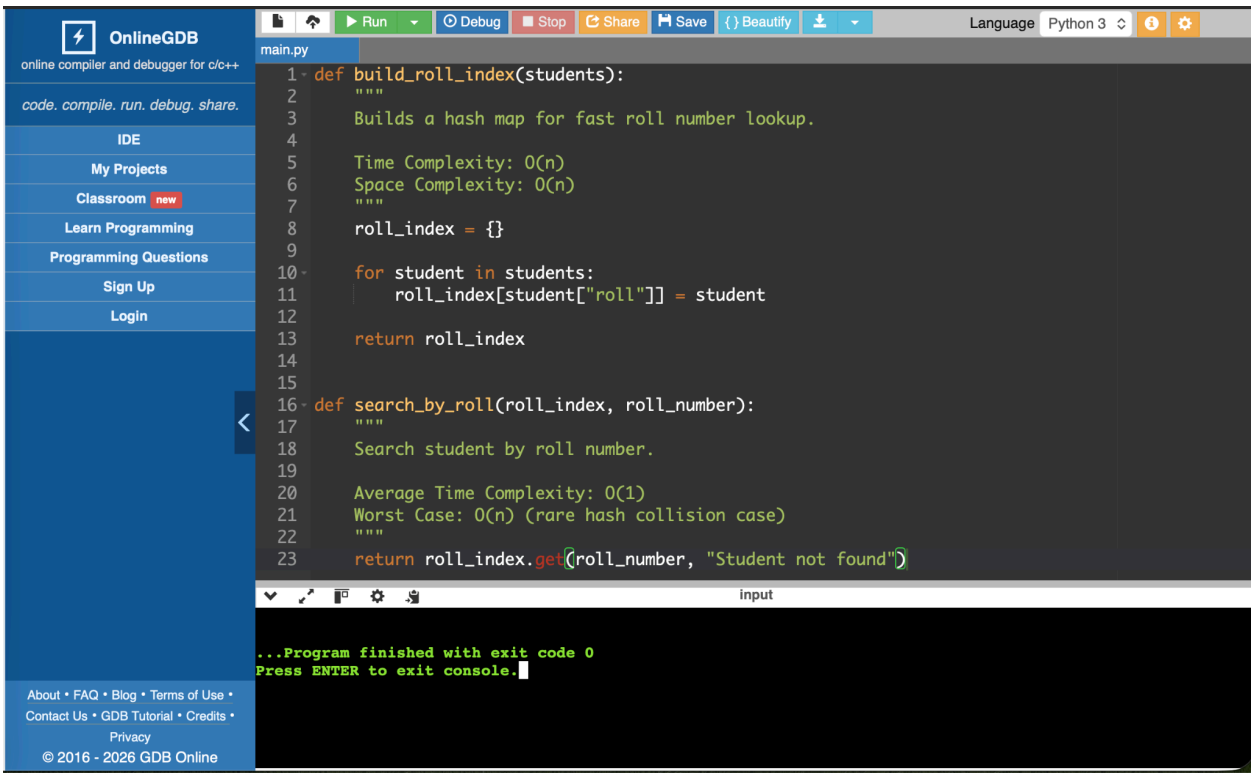
• Identify efficient searching and sorting algorithms using AI assistance.
• Justify the choice of algorithms.
• Implement the algorithms in Python.

Code

```
        def build_roll_index(students):
    """
    Builds a hash map for fast roll number lookup.

    Time Complexity: O(n)
    Space Complexity: O(n)
    """
    roll_index = {}

    for student in students:
        roll_index[student["roll"]] = student
```

```
    return roll_index


def search_by_roll(roll_index, roll_number):
    """
    Search student by roll number.

    Average Time Complexity: O(1)
    Worst Case: O(n) (rare hash collision case)
    """
    return roll_index.get(roll_number, "Student not found")
```





ask Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

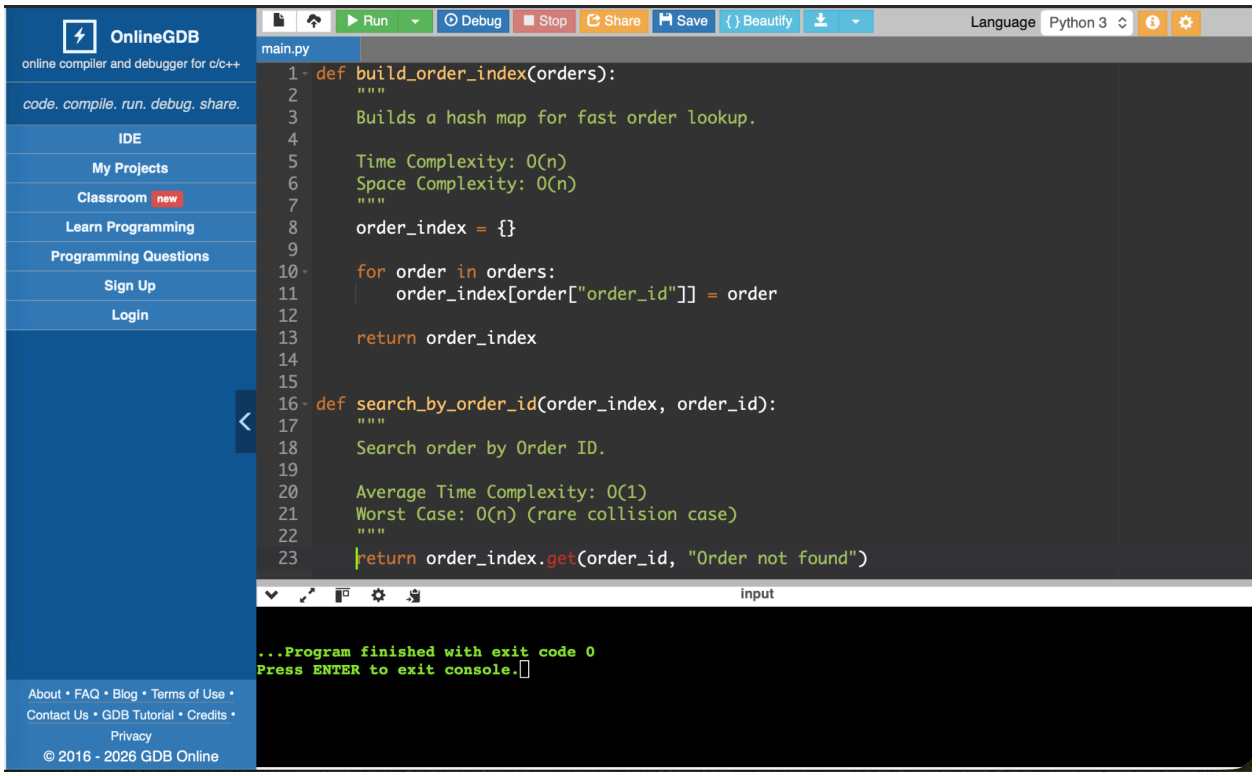
1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.

Student Task

• Use AI to suggest optimized algorithms.
• Justify the algorithm selection.
• Implement searching and sorting modules in Python.

Code

```
def build_order_index(orders):
    """
    Builds a hash map for fast order lookup.

    Time Complexity: O(n)
    Space Complexity: O(n)
    """
    order_index = {}

    for order in orders:
        order_index[order["order_id"]] = order

    return order_index


def search_by_order_id(order_index, order_id):
    """
    Search order by Order ID.

    Average Time Complexity: O(1)
    Worst Case: O(n) (rare collision case)
    """
    return order_index.get(order_id, "Order not found")
```


OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
About • FAQ • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online
Run
Debug
Stop
Share
Save
{ } Beautify
Language
Python 3
main.py
def build_order_index(orders):
"""
Builds a hash map for fast order lookup.
Time Complexity: O(n)
Space Complexity: O(n)
"""
order_index = {}
for order in orders:
order_index[order["order_id"]] = order
return order_index
def search_by_order_id(order_index, order_id):
"""
Search order by Order ID.
Average Time Complexity: O(1)
Worst Case: O(n) (rare collision case)
"""
return order_index.get(order_id, "Order not found")
input
...Program finished with exit code 0
Press ENTER to exit console.